Assignment 1 Report - Group 6

——— Adarsh Gupta (220101003) ——— Aditya Nanda (220101006) ———
——— Aryan Raj (220101019) ——— Ashish Raj (220101020) ———

CS344 Fall 2024

Task 1.1: Sleep

A new file sleep.c is created to implement the new user program, with the following code:

sleep.c

```c
/*
    User Program Name: sleep
    Usage:  sleep NUMBER
            or:  sleep OPTION
        Pause for NUMBER ticks.
        OPTION can be:
            -help        display help and exit
            -version     output version information and exit
    System Call Dependencies: Uses sleep system call
*/

#include "types.h" // contains typedef for common data types
#include "user.h" // contains system call templates avalaible for use in user programs

// These are additional utility functions implemented below
void printHelpAndExit(void);
void printVersionAndExit(void);
void printErrorAndExit(int);
int isNumber(const char*);

int main(int argc, char *argv[]){
    // Checking number of arguments passed through Command Line, and raising appropriate errors if any discrepency found
    if(argc == 1 || argc > 2) printErrorAndExit(argc);

    // Checking if OPTION's are used
    if(!strcmp(argv[1], "-help")) printHelpAndExit();
    if(!strcmp(argv[1], "-version")) printVersionAndExit();

    // Checking if a number is passed as input (this error checking is not implemented in atoi)
    if(!isNumber(argv[1])) printErrorAndExit(0);

    // Sleeping for specified number of ticks
    int numTicksToSleep = atoi(argv[1]);
    sleep(numTicksToSleep);

    exit();
}

void printHelpAndExit(void){
```

```
    printf(0, "\nUsage: sleep NUMBER\n   or:  sleep OPTION\nPause for NUMBER ticks. \nOPTION can be:\n
    -help        display this help and exit\n   -version     output version information and exit\n");
    exit();
}

void printVersionAndExit(void){
    printf(0, "Version 1.0\n (you really need another version of this?)\n");
    exit();
}

void printErrorAndExit(int argc){
    printf(1, "sleep: %s\nTry 'sleep -help' for more information.\n", argc == 1? "missing operand" :
    "incorrect usage");
    exit();
}

int isNumber(const char * s){
    // Iterates through the characters, and returns 0 if any character is not a digit
    while(*s != '\0'){
        if(!('0' <= *s && *s <= '9')) return 0;
        s++;
    }
    return 1;
}
```

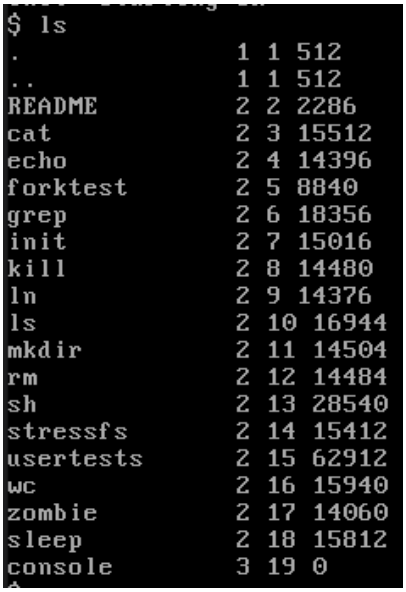
Makefile

To add the user program, to the list of user programs available, the following changes needed to be made to Makefile:

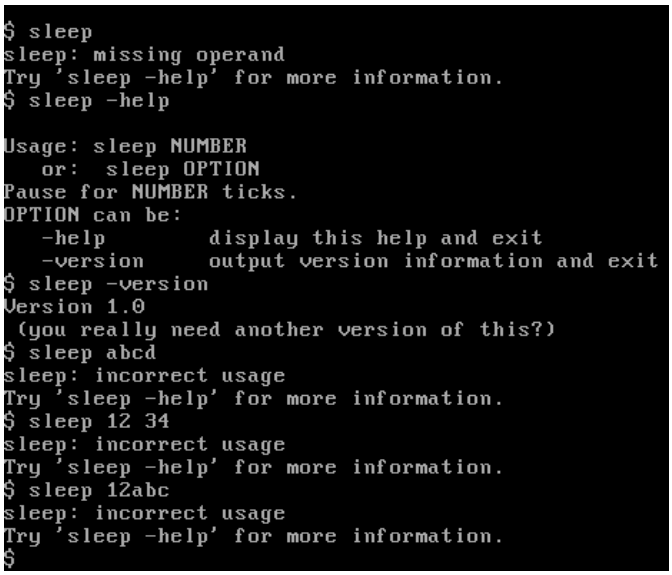
```
EXTRA=\
     mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
     ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
     printf.c umalloc.c sleep.c\ // Modified EXTRA here
     README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
     .gdbinit.tmpl gdbutil\
```

```
UPROGS=\
     _cat\
     _echo\
    ...
    _zombie\
     _sleep\ // Added compiled binary link to UPROGS
```

Git Patch: https://drive.google.com/drive/folders/1z5TX2cQaXB6wQDyPs9cYlXr0IVcqE4OD
Output:



(a) ls Output



(b) sleep General Program Usage

Figure 1: (a) ls output and (b) sleep usage.



(a) Sleep run command



(b) After 100 ticks

Figure 2: sleep Working

Task 1.2: Animations!

There are 2 files created to implement this new user program, namely bunny.c and bunny.h.

bunny.h

This file contains the global character 2D array, animation. The animation array stores the entire ASCII animation sequence. The ascii animation is made as a global variable to avoid stack overflow.

```
const char animation[37][1508] = {
    DATA
};
```

bunny.c

This file contains the implementation of the state machine given in the assignment.

```
/*
    User Program Added: bunny
    Role: plays a continously looping person swaying their head
*/

#include "types.h" // contains typedef for common data types
#include "user.h" // contains system call templates avalaible for use in user programs
#include "bunny.h" // contains the animation ascii art array

int main(void){
```

```
    int counter = 0;

    for(;;){
        // Reposition the cursor to top of screen and clear the screen
        printf(1, "\x1b[H\x1b[J");

        // Display Current frame
        printf(1, "%s", animation[counter]);

        // Sleep
        sleep(2);

        // Increment counter to go to next frame
        counter = (counter+1)%37;
    }
}
```

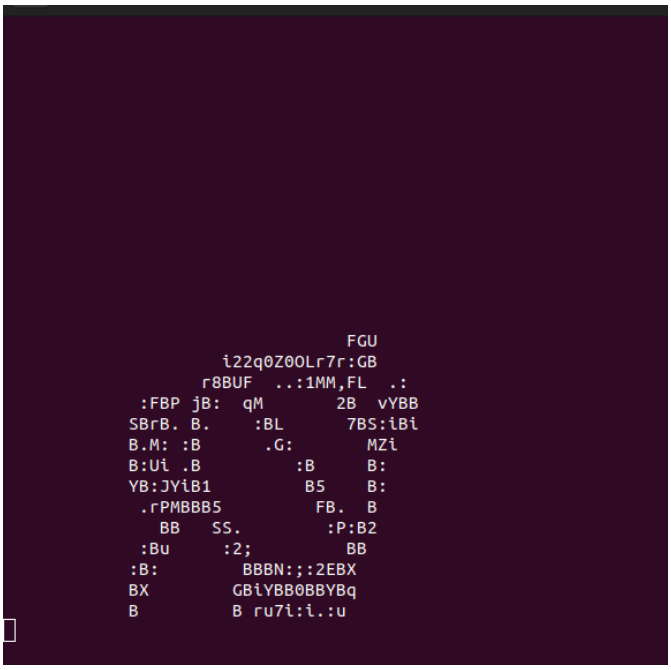
Makefile

To add the user program, to the list of user programs available, the following changes needed to be made to Makefile:

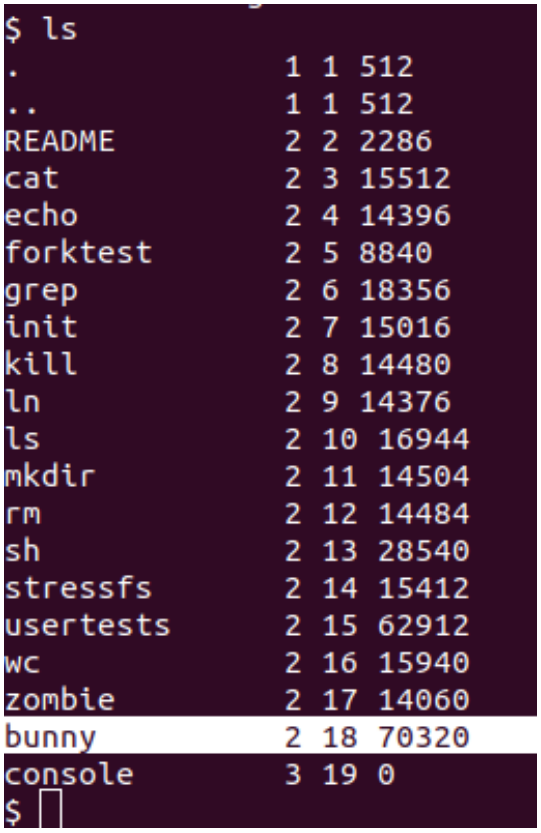
```
EXTRA=\
     mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
     ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
     printf.c umalloc.c bunny.c\ // Modified EXTRA here
     README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
     .gdbinit.tmpl gdbutil\
```

```
UPROGS=\
     _cat\
     _echo\
    ...
     _zombie\
     _bunny\ // Added compiled binary link to UPROGS
```

Git Patch: https://drive.google.com/drive/folders/1J2x4LiIa773kJUmPRC0hZa0ACudI-f8W
Output:



(a) One of 37 frames of a Rabbit Dancing



(b) ls Output

Figure 3: sleep Working

Task 1.3: Statistics

The objective of this assignment was to extend the wait system call in xv6 to create a new system call wait2, which provides additional information about the process's execution time, including:

1. Ready Time (retime): The total number of clock ticks during which the process was ready but not running.
2. Running Time (rutime): The total number of clock ticks during which the process was actively running.
3. Sleeping Time (stime): The total number of clock ticks during which the process was waiting for I/O.

These statistics are populated in the trap function, which is executed at every timer interrupt.

To store the statistics in the process struct and populate them while the child process executes, we had to modify the following files:

1. proc.h The proc structure was updated to include fields for tracking the times for each process.

```
// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
    ...
  char name[16];               // Process name (debugging)
  uint ctime;                   // Creation time of process
```

```
  uint stime;                  // Time spent by process in SLEEPING state
  uint retime;                 // Time spent by process in READY state
  uint rutime;                 // Time spent by process in RUNNING state
};

```

2. fork(void) function call in proc.c The fork function was updated to initialise the ctime, retime, rutime, and stime fields when a new process is added to ready queue.

```
int
fork(void)
{
  int i, pid;
    ...
  np->state = RUNNABLE;

  // Initialize Statistics related fields
  np->ctime = ticks;
  np->stime = 0;
  np->retime = 0;
  np->rutime = 0;

  release(&ptable.lock);
  return pid;
}

```

3. Updating the Trap Code We added a function call, to increase the statistics values' at every timer interrupt.

```
void
trap(struct trapframe *tf)
{
  if(tf->trapno == T_SYSCALL){
        ...
      release(&tickslock);
    }

    update_times(); // Called at every timer interrupt

    lapiceoi();
    break;
        ...
  case T_IRQ0 + IRQ_IDE+1:
    // Bochs generates spurious IDE1 in

```

4. update_time(void) function added in proc.c This function called at every timer interrupt, increases the statistics value for all processes.

```
void update_times(void){
  struct proc *p;

  // Acquire the process table lock to prevent race conditions
  acquire(&ptable.lock);

  // Iterate over all processes in the process table
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
```

```
    if(p->state == SLEEPING) {
      p->stime++;  // Increment sleeping time
    } else if(p->state == RUNNABLE) {
      p->retime++;  // Increment ready time
    } else if(p->state == RUNNING) {
      p->rutime++;  // Increment running time
    }
  }

  // Release the process table lock
  release(&ptable.lock);
}

```

To implement the system call wait2, which aggregates the statistics from a child process, the following files were modified.

1. syscall.h Defined the index of sys_wait2 in the kernel system call table.

```
#define SYS_wait2  22  // New system call for wait2
```

2. syscall.c We added the function template of the system call, and added the system call to the kernel system call table.

```
extern int sys_wait2(void); // Add this line
```

```
[SYS_wait2]   sys_wait2,  // Add this line //a2 code
```

3. usys.S We need to define a user wrapper for the system call, so that a user program can actually make this system call.

```
SYSCALL(wait2)
```

4. user.h Add the user system call template

```
int wait2(int *retime, int *rutime, int *stime);
```

5. sysproc.c Add the system call code for wait2.

```
int
sys_wait2(void)
{
    int *retime, *rutime, *stime;

    // Fetch the three integer pointer arguments from the stack
    if (argptr(0, (char**)&retime, sizeof(int*)) < 0)
        return -1;
    if (argptr(1, (char**)&rutime, sizeof(int*)) < 0)
        return -1;
    if (argptr(2, (char**)&stime, sizeof(int*)) < 0)
        return -1;

    struct proc *curproc = myproc();
    int pid;

    acquire(&ptable.lock);
```

```
    while(1){
        struct proc *p;
        int havekids = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->parent != curproc)
                continue;
            havekids = 1;
            if(p->state == ZOMBIE){
                // Found a zombie child, collect its statistics
                *retime = p->retime;
                *rutime = p->rutime;
                *stime = p->stime;
                pid = p->pid;
                kfree(p->kstack);
                p->kstack = 0;
                freevm(p->pgdir);
                p->pid = 0;
                p->parent = 0;
                p->name[0] = 0;
                p->killed = 0;
                p->state = UNUSED;
                release(&ptable.lock);
                return pid;
            }
        }

        // No point waiting if we don't have any children
        if(!havekids || curproc->killed){
            release(&ptable.lock);
            return 1;
        }

        sleep(curproc, &ptable.lock);  // Wait for a child to exit
    }
}

```

Added a user program to get the statistics. To achieve this, the following changes were made:

1. test_wait2.c This is the new user program added

```
#include "types.h"
#include "stat.h"
#include "fcntl.h"
#include "user.h"

int main(void) {
    int retime, rutime, stime, pid1, pid2;

    if((pid1 = fork()) == 0){
        for(volatile int i = 0; i < 100; i++){
            int fd = open("README", O_RDONLY);
            asm("nop");
            if(fd>=0){
                close(fd);
            }
        }
        exit();
    }

```

```
    if((pid2 = fork()) == 0){
        for(volatile int i = 0; i < 100; i++){
            sleep(10);
        }
        exit();
    }

    wait2(&retime, &rutime, &stime);
    printf(1, "Child PID: %d\n", pid1);
    printf(1, "Ready Time: %d ticks\n", retime);
    printf(1, "Running Time: %d ticks\n", rutime);
    printf(1, "Sleeping Time: %d ticks\n", stime);

    printf(0, "\n");

    wait2(&retime, &rutime, &stime);
    printf(1, "Child PID: %d\n", pid2);
    printf(1, "Ready Time: %d ticks\n", retime);
    printf(1, "Running Time: %d ticks\n", rutime);
    printf(1, "Sleeping Time: %d ticks\n", stime);

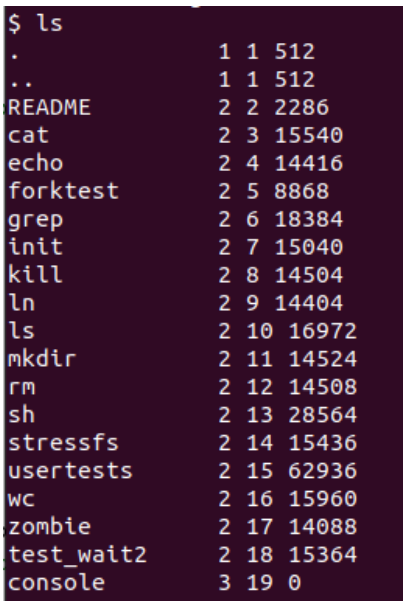
    exit();
}
```

2. Makefile To add the new user program to the list of user programs avalaible.

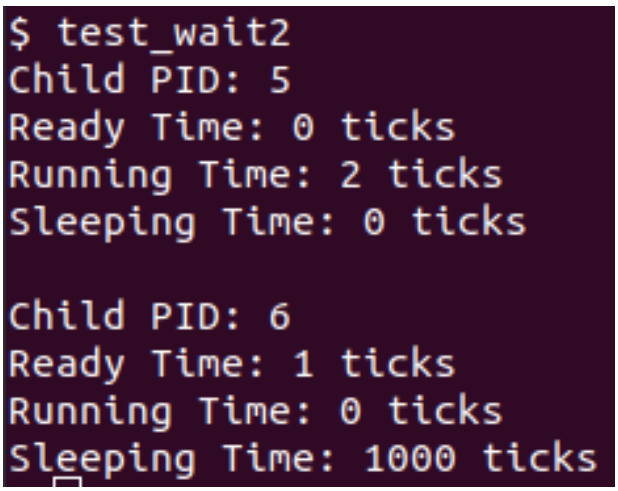
```
    _test_wait2\ // Added to UPROGS
```

```
    test_wait2.c\ // Added to Extras
```

Git Patch: https://drive.google.com/drive/folders/12oKkL9JVSu-cPe18dFJzCCNHRp2KheUA
Output:



(a) ls Output



(b) test_wait2 Output

Figure 4: sleep Working